

Assignment 2 COL724/7524

Part A: Link Scheduling

Overview

Implement a multithreaded client-server system in C++ using POSIX sockets. The server must handle concurrent clients, support transferring and reading text files, maintain a request dispatching mechanism, and implement selectable scheduling policies (`fcfs`, `sjf`, `rr`). The implementation must also produce per-request timing metrics (arrival, start, finish) and aggregate statistics to enable performance comparison across scheduling policies and configurations.

Configuration File

Both server and client must read a configuration file named `config.json` at startup. The JSON schema to be used is:

```
{
  "server_ip": "127.0.0.1",
  "server_port": 9000,
  "server_threads": 4,
  "client_threads": 8
}
```

Field descriptions:

- `server_ip`: IP address where the server binds and listens (string).
- `server_port`: TCP port number for the server (integer).
- `server_threads`: Number of worker threads the server creates (integer).
- `client_threads`: Number of concurrent client instances used in experiments (integer).

Command-Line Arguments

The following runtime parameters must be supplied via command-line arguments to the server executable:

- `--sched <policy>` Scheduler policy to use: `fcfs`, `sjf`, or `rr`.
- `--quantum <Q>` Time quantum for Round-Robin scheduling (integer). Required when `--sched rr`.
- `--file <filename>` Input dataset filename or directory of text files.
- `--p <N>` Packetization parameter tells the server how many lines of text (from the file being transferred) it should group together and send in one “packet” (i.e., one `send()` call).

Client API

Clients must be able to interact with the server using the following operations:

- **PUT** <filename>: Upload a text file to the server.
- **GET** <filename>: Download a text file from the server.

The details of how file contents are transmitted are left to the students. The requirement is only that text files can be successfully transferred between clients and the server using the above operations.

Requirements

1. The server must be multithreaded and correctly synchronized to handle concurrent requests from multiple clients.
2. The server must implement selectable scheduling policies `fcfs`, `sjf`, and `rr`. The active policy is chosen via the `--sched` command-line argument. Round-Robin requires the `--quantum` argument.
3. The server must record per-request timestamps: arrival time, service start time, and finish time using a monotonic clock. These timestamps must be used to compute response time and waiting time statistics.
4. The server and client must both read the configuration file `config.json` at startup to obtain network and concurrency parameters.
5. The command-line arguments described above must be accepted and used.
6. Any required command-line argument or required JSON field missing at program start must produce a clear error message and exit.
7. The implementation must be robust with respect to malformed requests and must return an appropriate error message.
8. Internal implementation choices are left to the student.

Deliverables

Submit a single archive named as specified by course instructions. The archive must contain a top-level directory with the following:

1. **Source code:** all source files, a `Makefile` or build script, and any auxiliary scripts needed to run experiments. The server and client must compile on a standard Linux environment using `g++` with `-std=c++17` and `-pthread`.
2. **README:** concise build and run instructions that describe how to start the server and clients, the accepted command-line arguments, the meaning of configuration fields, and how to reproduce the experiments.
3. **Report (PDF):** a report describing implementation details, the experiments performed, and analysis. The report must include:

- Description of scheduling policies implemented.
 - Experimental setup (server/client threads, values of `--p`, workload characteristics).
 - Plots comparing performance (response time, throughput, waiting time) across the three scheduling policies.
 - Brief discussion of results and observed trade-offs.
4. **Graphs:** include plot image files (PNG or PDF) used in the report.
 5. **Sample input files / test files:** any text files used in experiments and any client command files used for tests.

You should submit a single zipped folder containing all the code as well as the report. Name it:

```
<entry_no.1>_<entry_no.2>_A.zip
```

Measurements and Experiments

- Measure response time and throughput for each scheduling policy under varying load levels. At minimum:
 - Fix server thread count and vary client thread count.
 - Fix client thread count and vary server thread count.
 - Vary packetization parameter `--p` and observe its effect.
- For each experiment, collect per-request arrival/start/finish timestamps and compute mean and median response times, mean waiting time, and throughput (requests per second).
- Include plots in the report that compare these metrics across scheduling policies.

Submission Checklist

Ensure the following before submission:

- Build completes using the provided `Makefile`.
- `README` clearly explains how to run the server and client with the configuration file and CLI flags.
- Report (PDF) and plots are included in the top-level directory.
- Both PUT and GET client operations are implemented and tested.

Part B: Load Balancer

Overview Implement a load balancer (LB) that accepts client connections and forwards client requests to four independent multithreaded server instances. The server

API remains the same as in Part A: clients must be able to **PUT** <filename> (upload a text file) and **GET** <filename> (download a text file). Internal design choices are left to the student.

Required behaviour

1. The load balancer must accept client connections and serve all client operations (PUT and GET) using the same client API as the Part A servers. From the client perspective, interacting with the LB should provide the same semantics as interacting with a server.
2. The LB must forward requests to **four** backend server instances. Each backend is an independent multithreaded server (running on a distinct port).
3. The LB must be concurrent and scalable so it can handle multiple client threads.
4. The LB must log operational metrics such as requests forwarded to each backend.
5. **Health checks:** the LB must send a short health-check message to each backend once every second. Each backend must reply to the health-check message. The LB must record each backend's health-check response (including observed round-trip time) into a log file for later analysis.

Algorithms and Design Choice You must implement any two distinct load-balancing algorithms of your choice using the health-check data collected by the load balancer. In the report, clearly describe both algorithms, explain why you chose them, and compare their performance through experiments. Your analysis should discuss the observed results, the trade-offs between the two algorithms, and justify your design decisions.

Configuration File For Part B, the configuration file must be named `config.json` and specify the LB's own address and the four backend servers. The schema is:

```
{
  "lb_ip": "127.0.0.1",
  "lb_port": 8000,

  "server1_ip": "127.0.0.1",
  "server1_port": 9001,

  "server2_ip": "127.0.0.1",
  "server2_port": 9002,

  "server3_ip": "127.0.0.1",
  "server3_port": 9003,

  "server4_ip": "127.0.0.1",
  "server4_port": 9004
}
```

Field descriptions:

- `lb_ip`, `lb_port`: IP address and port where the load balancer listens for client connections.
- `serverN_ip`, `serverN_port`: IP address and port for each backend server ($N = 1, 2, 3, 4$).

Deliverables (Part B)

1. Source code for the load balancer, build script / Makefile, and any helper scripts used to start backend servers for experiments.
2. README with build and run instructions, configuration format, and explanation of the health-check design.
3. A short report describing experimental setup and observations. Include plots or tables that use the LB health-check logs to show backend behaviour.
4. Health-check log files collected during experiments and any scripts used to produce the plots included in the report.

You should submit a single zipped folder containing all the code as well as the report. Name it:

`<entry_no.1>_<entry_no.2>_B.zip`

Notes for students

- Each backend must listen on a distinct port.
- The LB must perform health checks every second and record results in a log file.
- Students are free to design the load balancing algorithm and the format of the health-check messages exchanged with servers. Document choices clearly in the README.

Part C: Software-Defined Networking (SDN)

Part 1: Controller Hub and Learning Switch

You need to compare the performance of a Hub Controller and a Learning Switch.

- A Hub Controller redirects all the traffic on a switch to itself and then forwards it to all switch ports except the incoming port.
- A Learning Switch installs flow rules on the switches based on the MAC to Port mappings it learns from incoming traffic.

Begin by implementing a Controller Hub and Learning Switch. You are given a network topology file . Run the controller against the given network topology file and answer the following questions:

- Run `pingall` in both cases and report the installed rules in switches. Explain your observations briefly.

- Run a throughput test between h1 and h5 in both cases using `iperf`. Report the observed values. Explain the differences in the speed values between the Hub Controller and Learning Switch.

Hint: You can use the example code in the Ryu codebase. This part is relatively easy. You should use this to understand the working of Ryu controller.

Part 2: Shortest Path Routing

In this part, you will write a controller app that implements shortest path routing for a given network topology, while still performing L2 routing. The weights of the links should be set based on their capacities. You can test your code using a network topology that is provided with the assignment.

In your report, please briefly describe your approach and any assumptions you made.

Part 3 : Congestion-aware Shortest Path Routing

The shortest path described in Section 3 uses static link weights. In a real-world network, operators route traffic based on the current link load. Modify the shortest path routing app above to create dynamic routes based on the link utilization. Feel free to make assumptions whether you are routing at flow-level or packet-level. Just state them in the report along with your implementation logic. A proportion of the marks will be awarded based on the observed throughput for a given network topology.

Submission Instructions

Your submission should contain a single PDF (other formats will not be graded) called `report.pdf`.

In addition, you should submit the following files:

- **Part 1:** Attach the screenshots of the rules as well as the ping/iperf results in the PDF. In addition, submit two controller applications named `p1_learning.py` and `p1_hub.py`.
- **Part 2:** Submit the controller application named `p3_spr.py`.
- **Part 3:** Submit the controller application named `p4_ca_spr.py`.

You should submit a single zipped folder containing all the code as well as the report. Name it:

`<entry_no_1>_<entry_no_2>_C.zip`

where the entry numbers refer to you and your partner (if any).

Important Notes

- Before posting questions on Piazza, please make sure you have searched online and consulted available resources.

- If any part of the specification seems unclear or incomplete, make a reasonable assumption, clearly state it in your README or report, and proceed with your implementation.
- All submissions will be checked for plagiarism. Ensure that the work you submit is entirely your own.
- The assignment may be completed in groups of **at most two** students.
- Deadline for Part A: 10/10/2025
- Deadline for Part B: 15/10/2025
- Deadline for Part C: 25/10/2025